Luento 1

26.10.2026

Ohjelmiston elinkaari (software lifecycle)

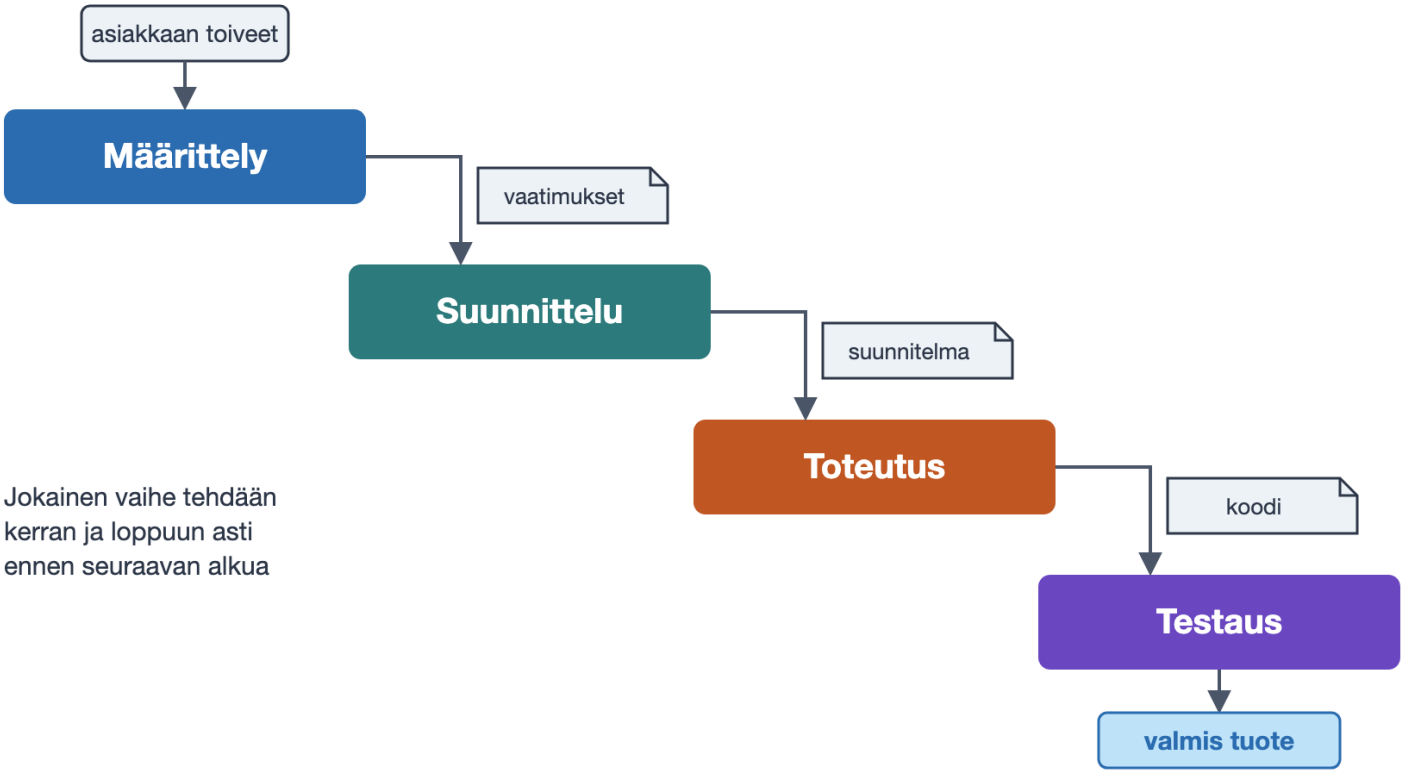
Riippumatta tyylistä ja tavasta, jolla ohjelmisto tehdään, käy ohjelmisto läpi seuraavat *vaiheet*

- Vaatimusten analysointi ja määrittely
- Suunnittelu
- Toteutus
- Testaus
- Ohjelmiston ylläpito ja evoluutio

Vaiheista muodostuu ohjelmiston “elinkaari”

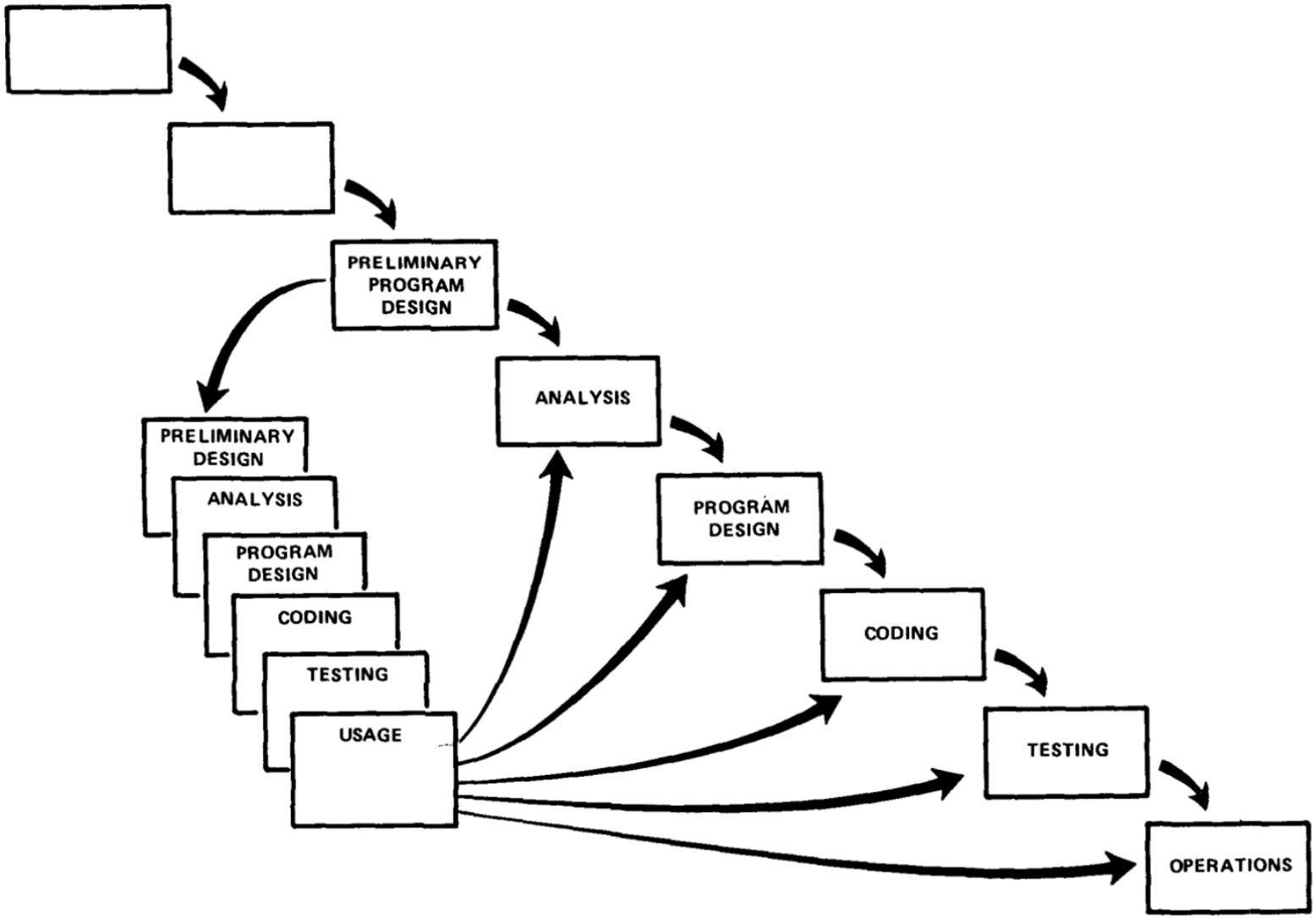
Vesiputousmalli

Winston W. Royce: Management of the development of Large Software, 1970

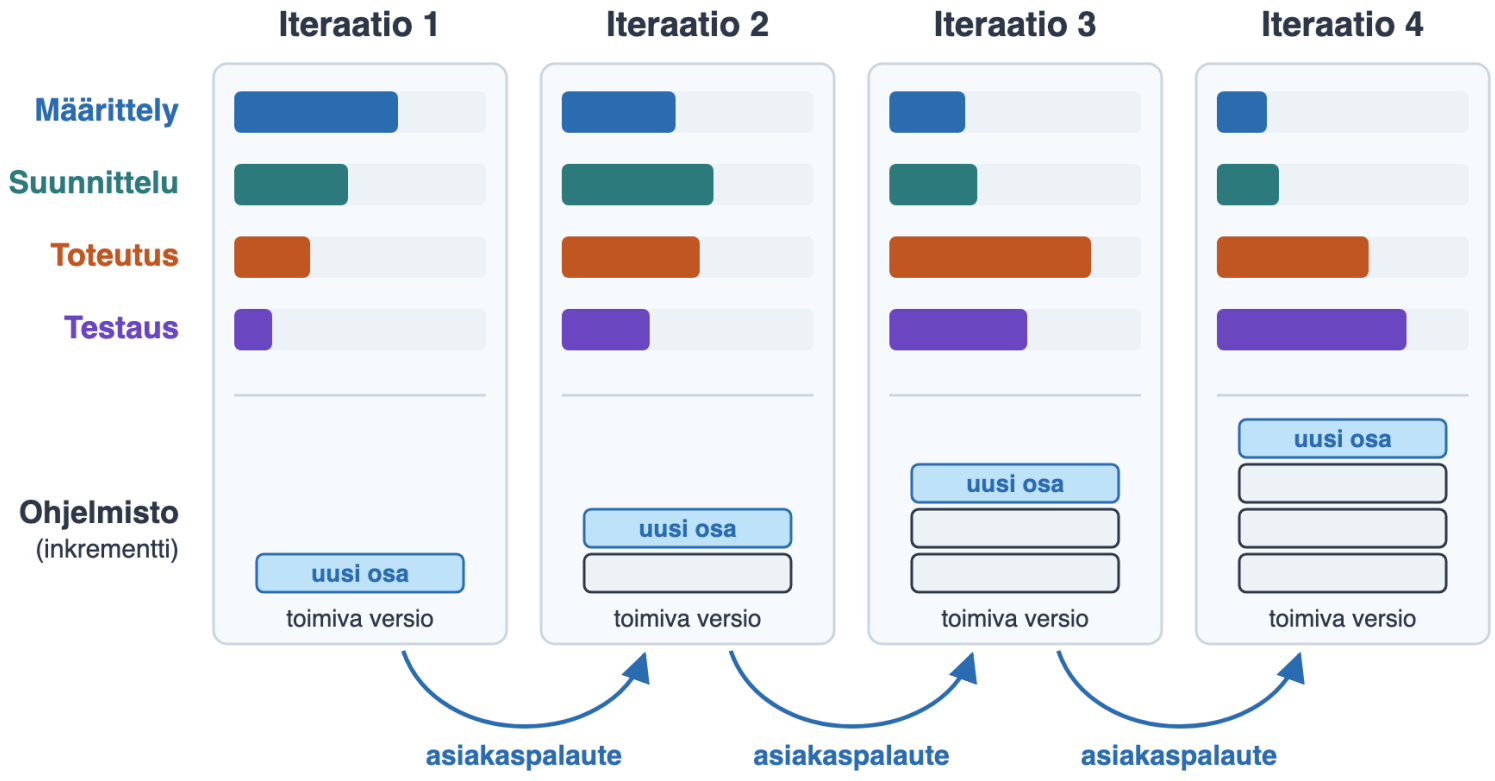


Asiakas näkee toimivan ohjelmiston vasta aivan projektin lopussa

Roycen kahden iteraation malli



Iteratiivinen ohjelmistokehitys



Jokaisen iteraation lopussa asiakas näkee toimivan version ja vaikuttaa seuraavan iteraation sisältöön

Ketterä manifesti 2001



Manifesto for Agile Software Development

We are uncovering better ways of developing
software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

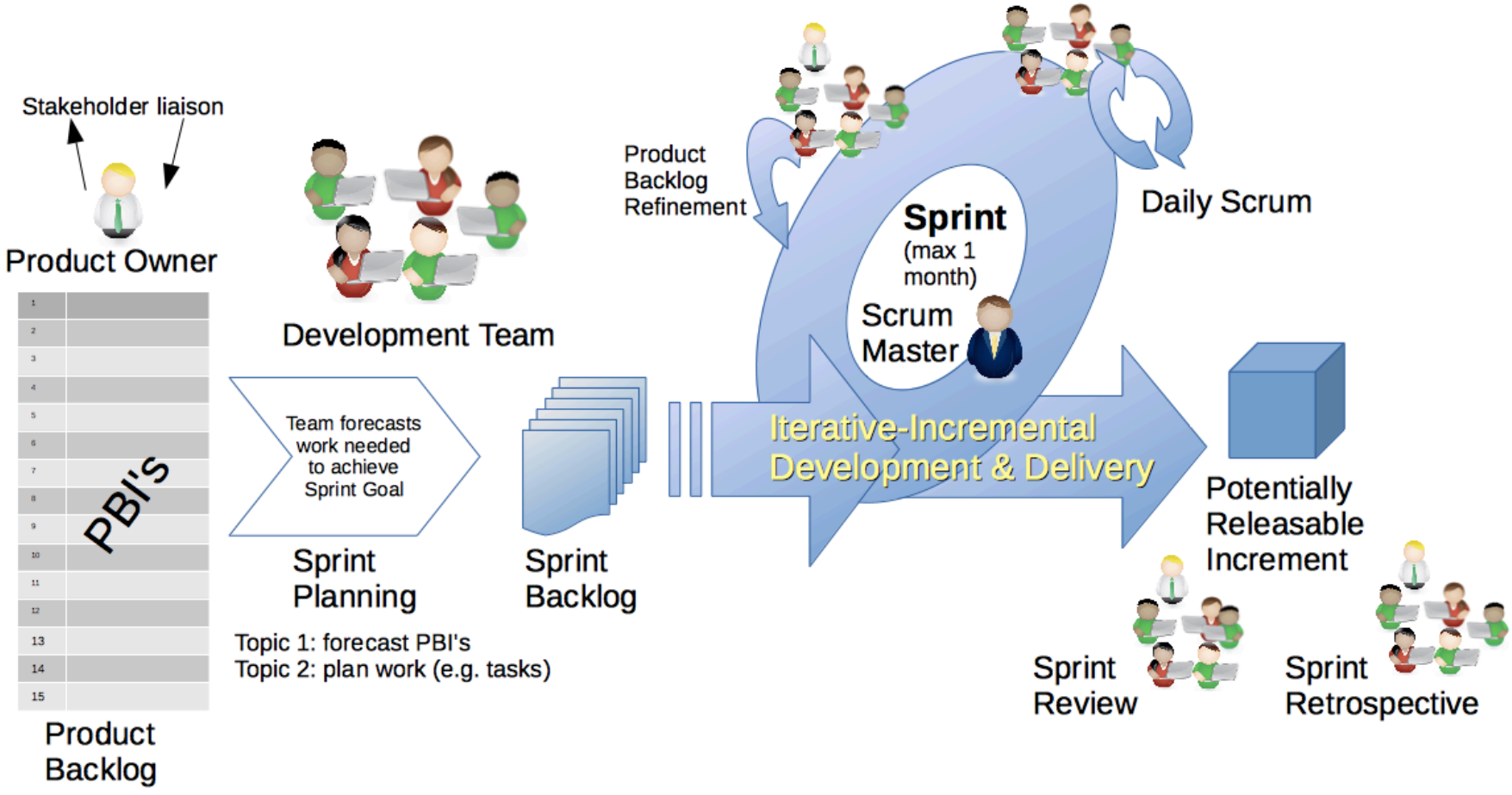
That is, while there is value in the items on
the right, we value the items on the left more.

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler
James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick
Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

Luento 2

27.10.2026

Scrum kuvana



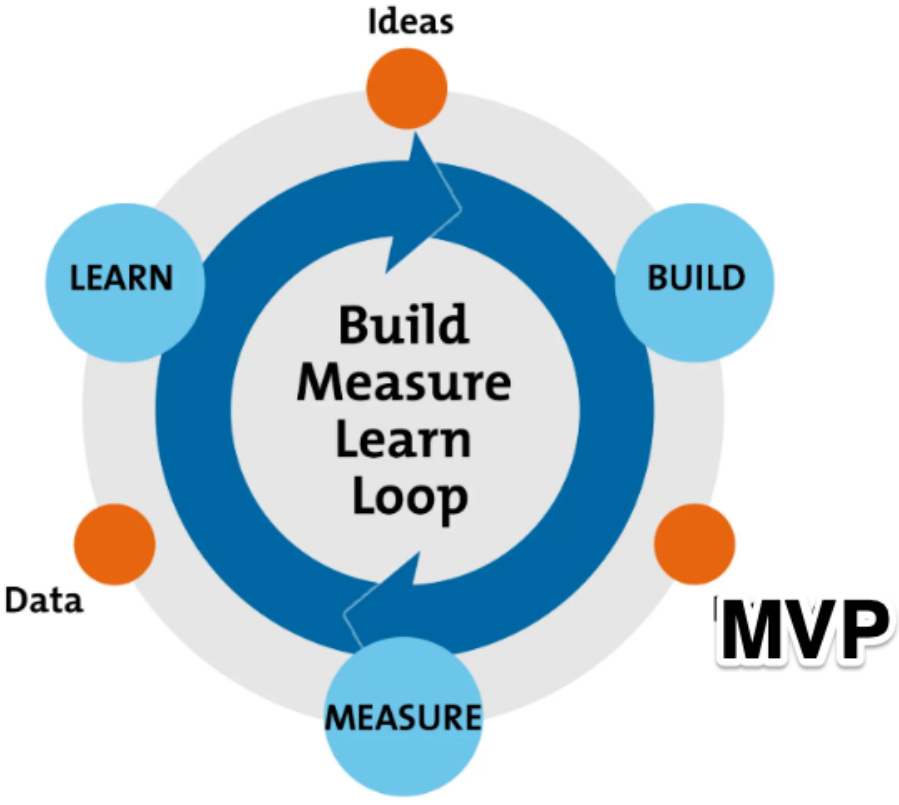
Luento 3

2.11.2026

Ohjelmiston elinkaari (software lifecycle)

- **Vaatimusten analysointi ja määrittely**
- Suunnittelu
- Toteutus
- Testaus
- Ohjelmiston ylläpito ja evoluutio

Vaatimusmäärittely 2010-luvulla: Lean startup



User story

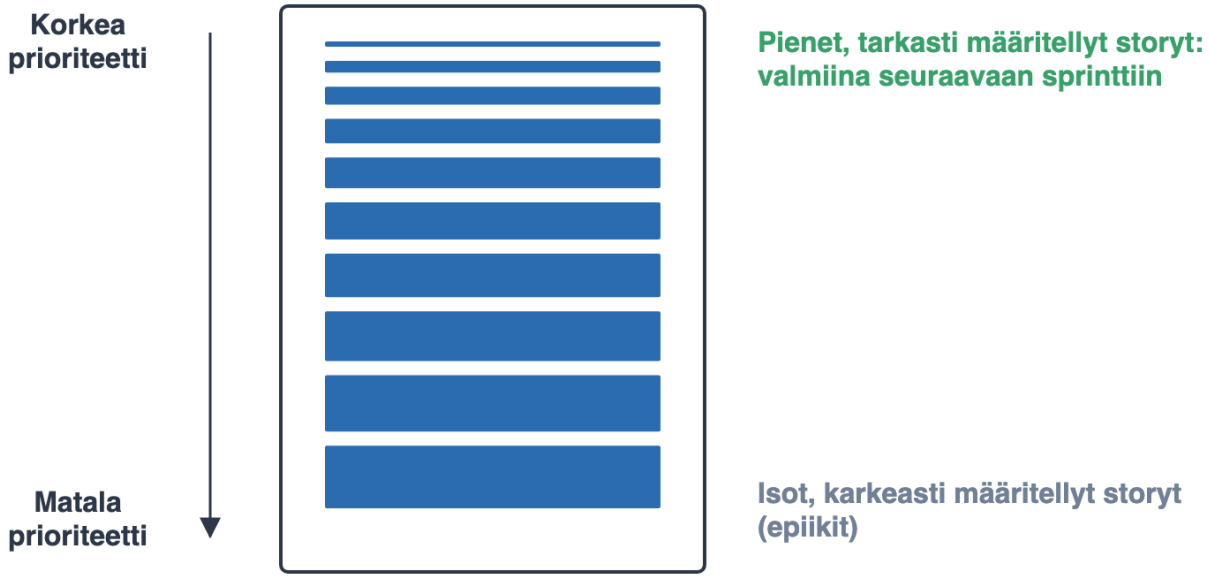
- Mike Cohn:
  - *User story kuvaa ohjelmiston käyttäjälle tai tilaajalle **arvoa tuottavan toiminnallisuuden**.*
- User storyssa on kolme osaa:
  1. **Kirjallinen kuvaus**
  2. **Keskustelut**
  3. **Testit**

Hyvän storyn kriteerit

- Bill Wake *INVEST in good User Stories*, kuusi toivottavaa ominaisuutta
  - Independent
  - Negotiable
  - Valuable to user or customer
  - Estimable
  - Small
  - Testable

Hyvä product backlog on DEEP

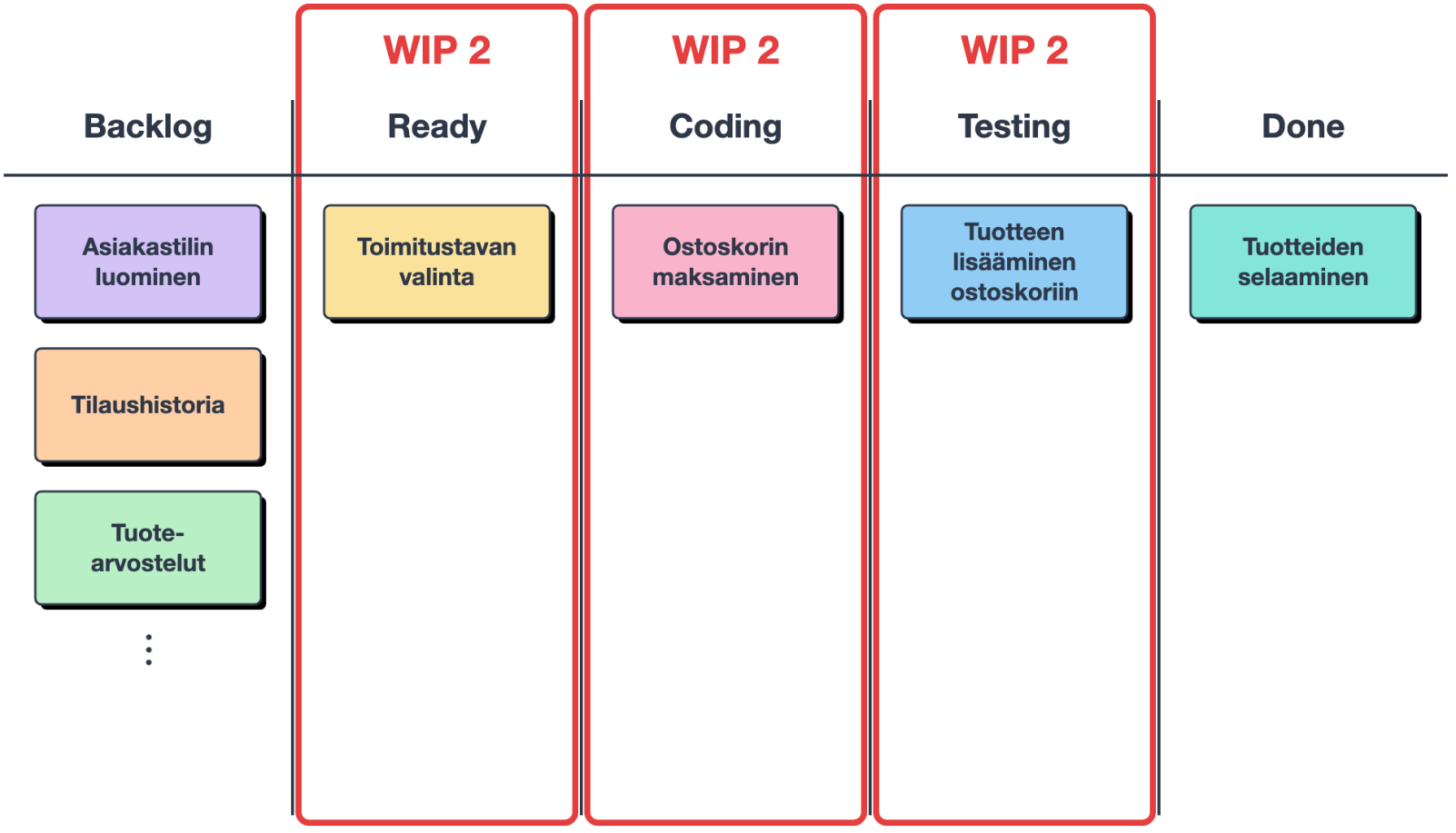
- Detailed appropriately
- Estimated
- Emergent
- Prioritized



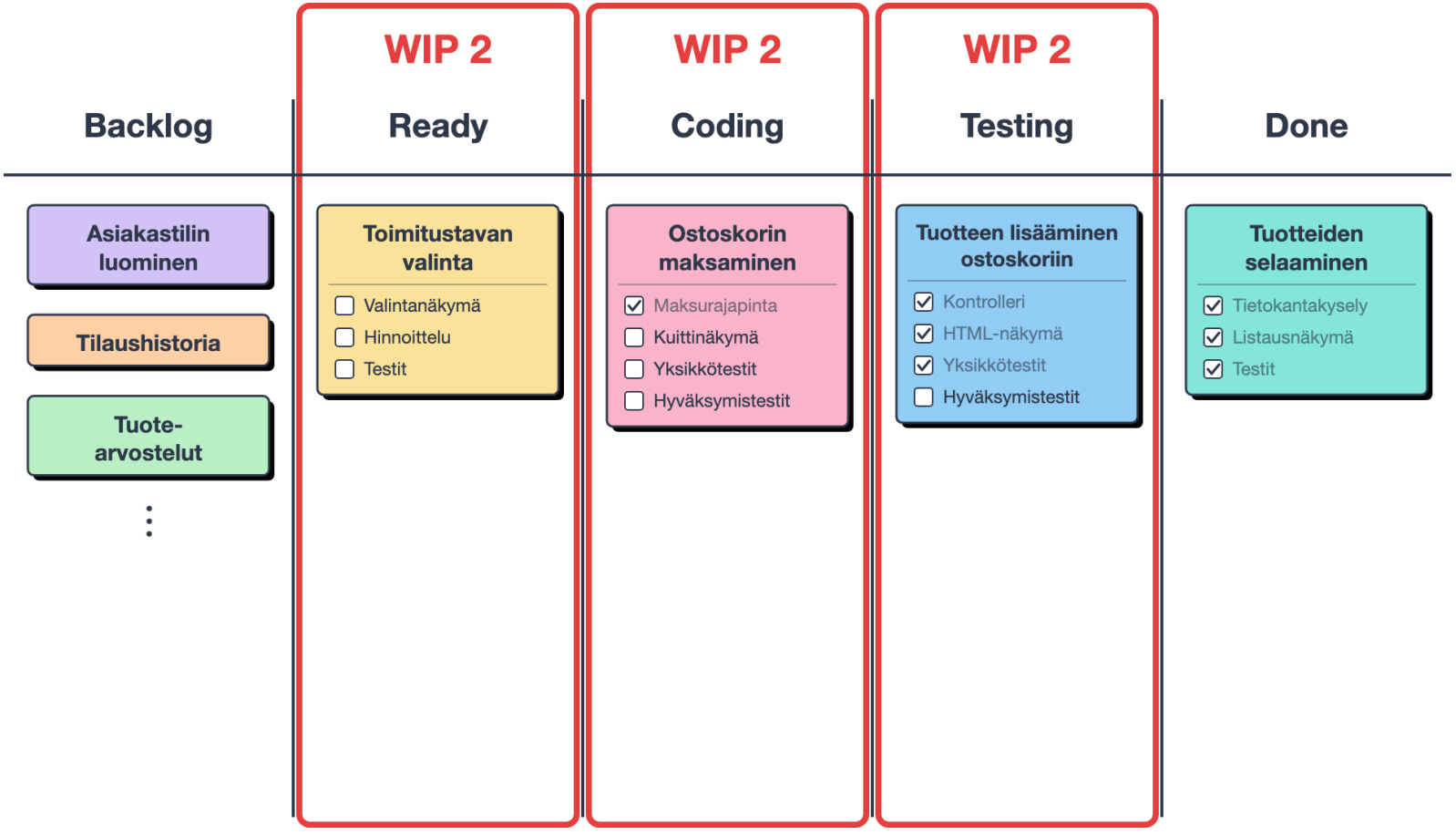
Luento 4

3.11.2026

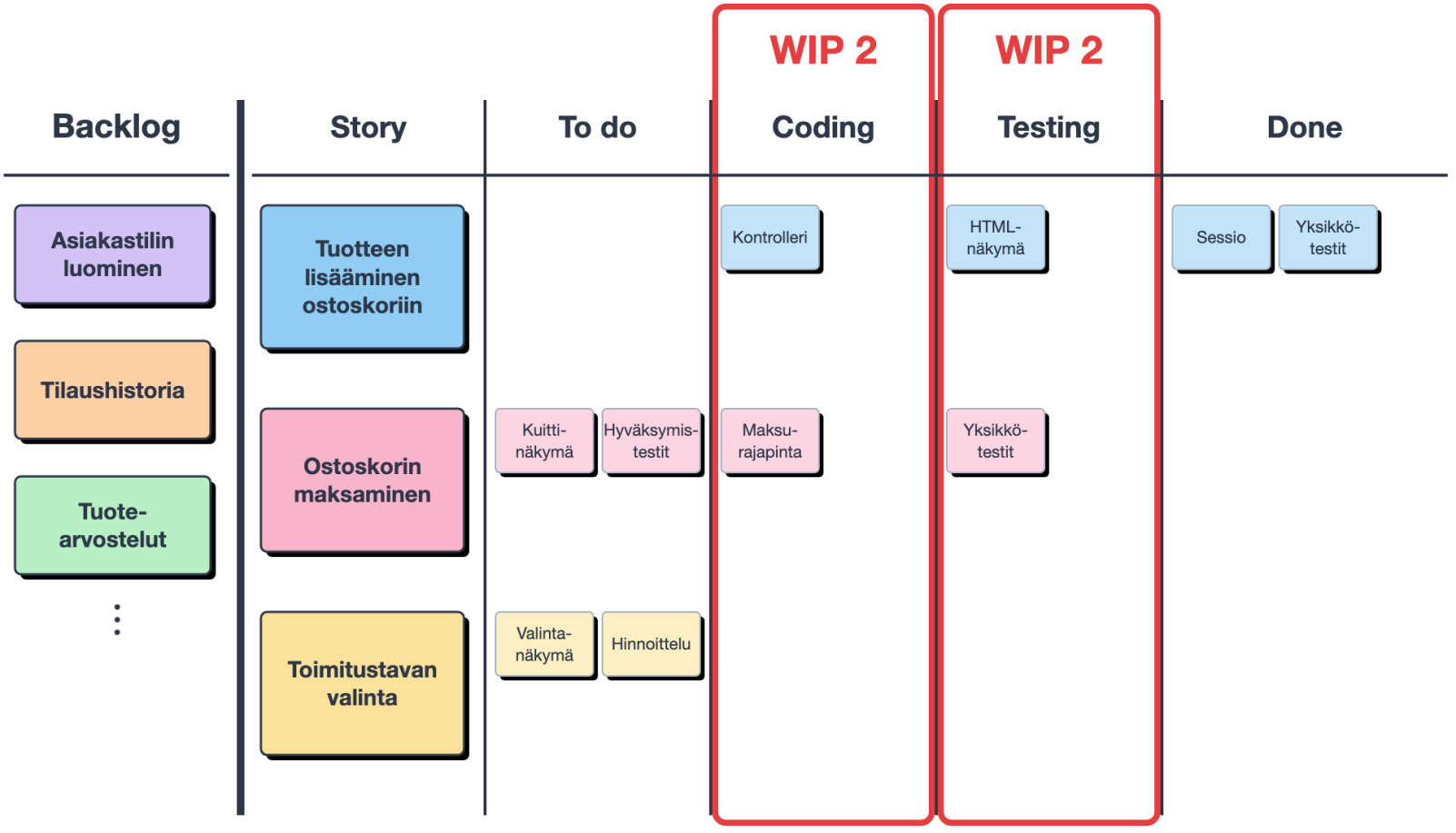
Scrumban: yksi kanban-taulu



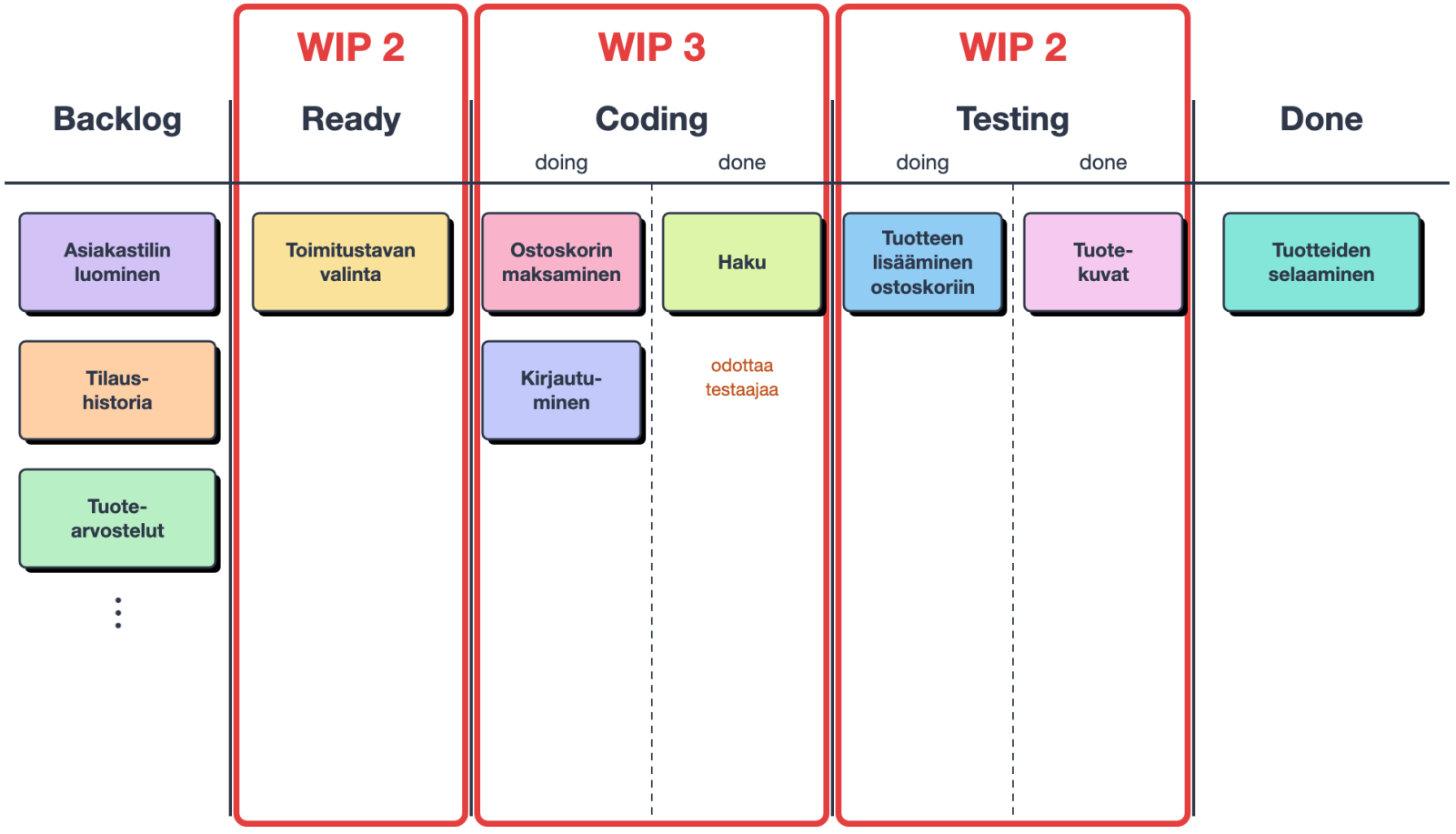
Scrumban: taskit storyn tarkistuslistana



Scrumban: storyt riveinä, taskit liikkuvat



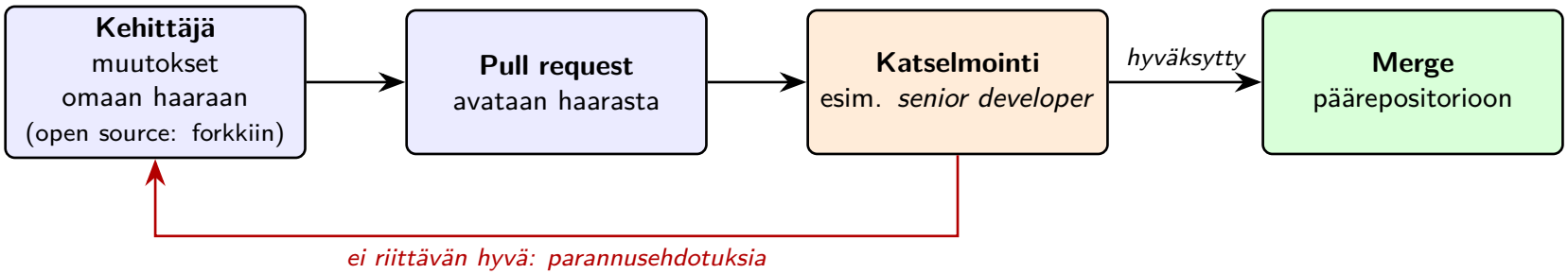
Scrumban: jaetut sarakkeet doing / done



Luento 5

9.11.2026

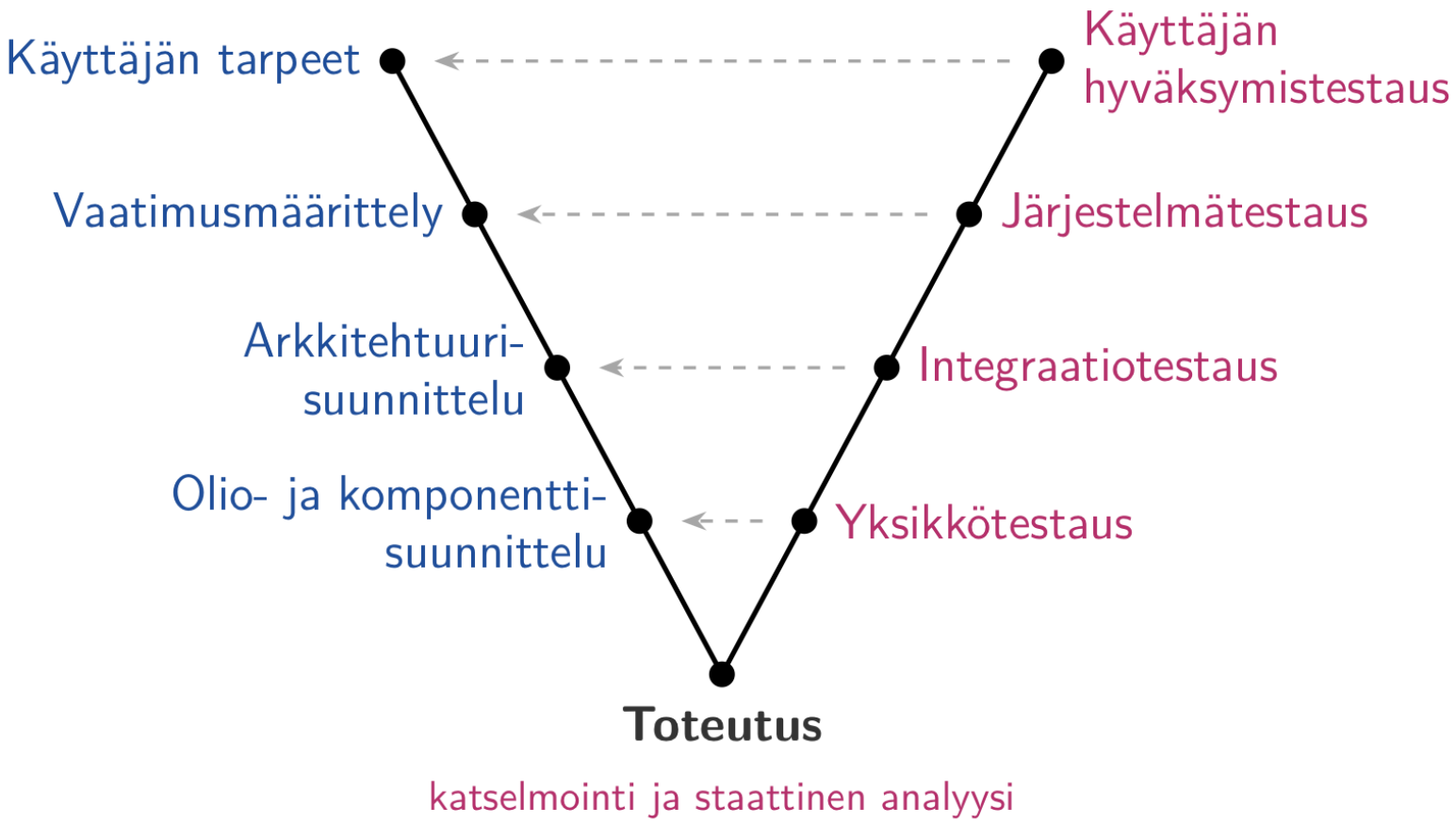
Koodin katselmointi: pull requestit



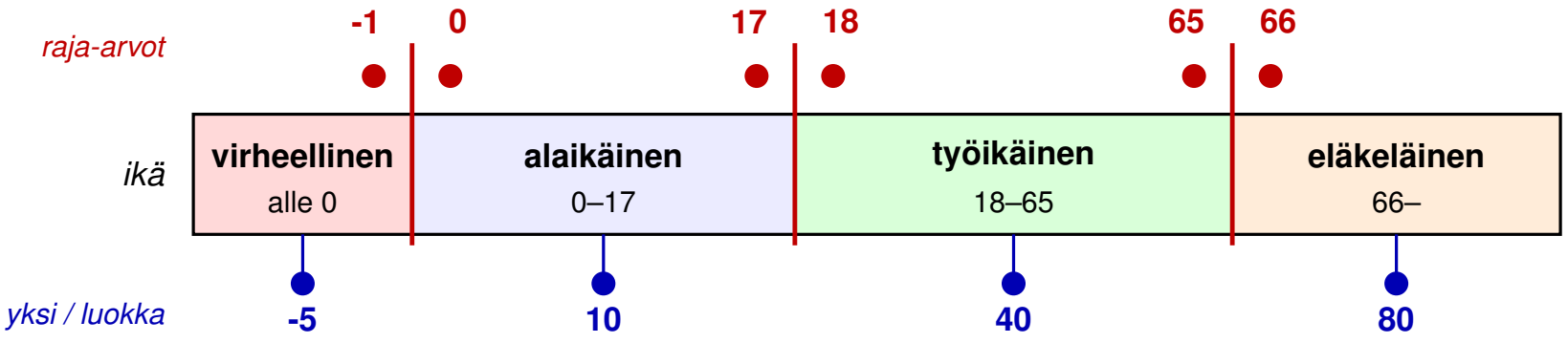
Testauksen tasot

- *Yksikkötestaus* (unit testing)
  - Yksittäisten luokkien, metodien ja moduulien testaus erillään muusta kokonaisuudesta
- *Integraatiotestaus* (integration testing)
  - Yksittäin testattujen komponenttien liittäminen yhteen eli integrointi ja kokonaisuuden testaus
- *Järjestelmätestaus* (system testing)
  - Toimiiko ohjelmisto vaatimuksiin kirjatulla tavalla?
  - Tutkii järjestelmää kokonaisuudessaan: *end to end -testaus*
  - Jakautuu useisiin alalajeihin
- *Käyttäjän hyväksymistestaus* (user acceptance testing)
  - Loppukäyttäjän tuotteelle suorittama testaus

“V-malli”

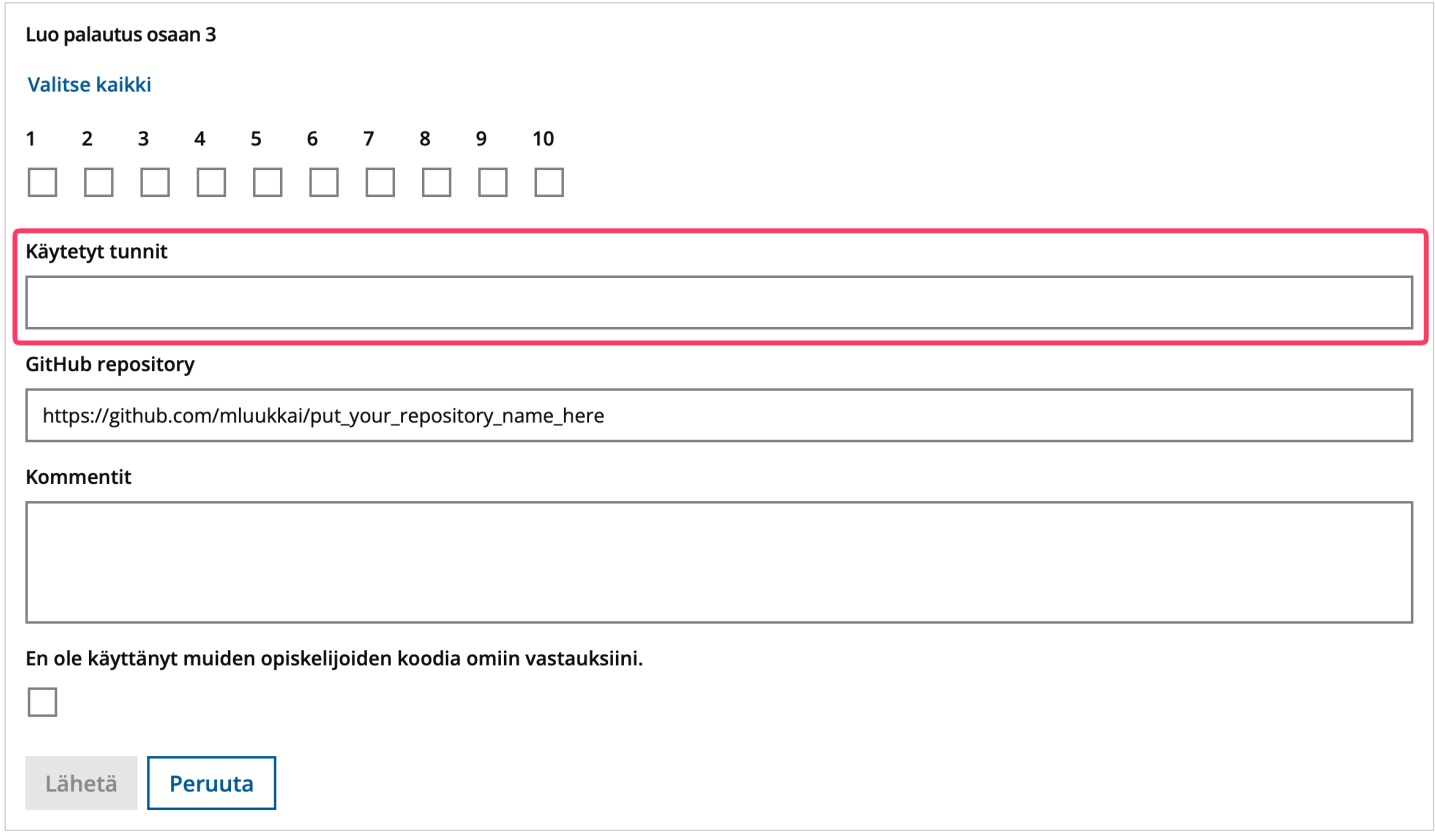

Käyttäjän tarpeet
Käyttäjän hyväksymistestaus
Vaatimusmäärittely
Järjestelmätestaus
Arkkitehtuuri-suunnittelu
Integraatiotestaus
Olio- ja komponentti-suunnittelu
Yksikkötestaus
Toteutus
katselmointi ja staattinen analyysi

Ekvivalenssiluokat ja raja-arvot



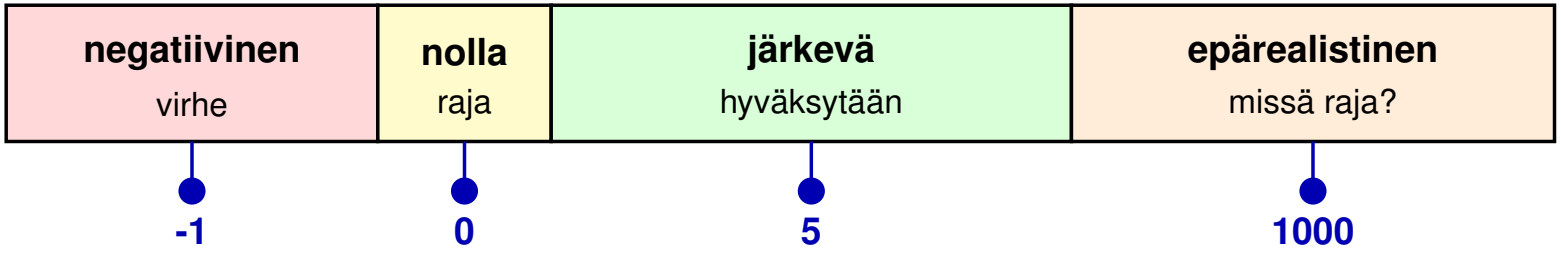
Testisyötteiden valinta: palautussovellus

- Mitä testitapauksia kannattaisi valita palautussovelluksen testaamiseen?

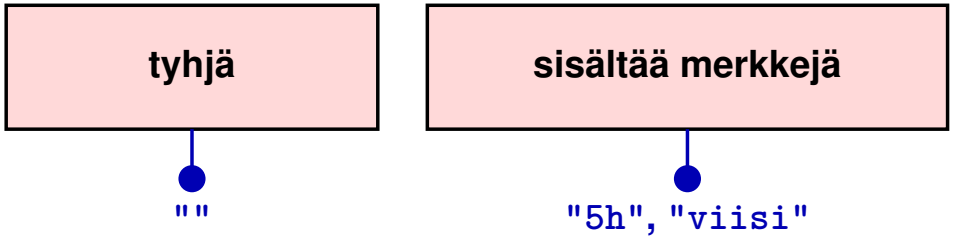


Testisyötteiden valinta: palautussovellus

Tunnit: numeeriset syötteet



Ei-numeeriset syötteet



Mutaatiotestaus

Koodi

```
def ikaluokka(ika):
  if ika < 18:
    return "alaikäinen"
  return "aikuinen"
```

Testit

```
assert ikaluokka(10) == "alaikäinen"
assert ikaluokka(40) == "aikuinen"
```

rivikattavuus 100 %

Puuttuva raja-arvotesti, tappaa mutantin 2

```
assert ikaluokka(18) == "aikuinen"
```

Mutantti 1

```
if ika > 18:
```

→ **testi hajoaa** tapettu

Mutantti 2

```
if ika <= 18:
```

→ **testit menevät läpi** elää!

Mutantti 3

```
return None
```

→ **testi hajoaa** tapettu

mutaatiopisteet 2/3 = 67 %

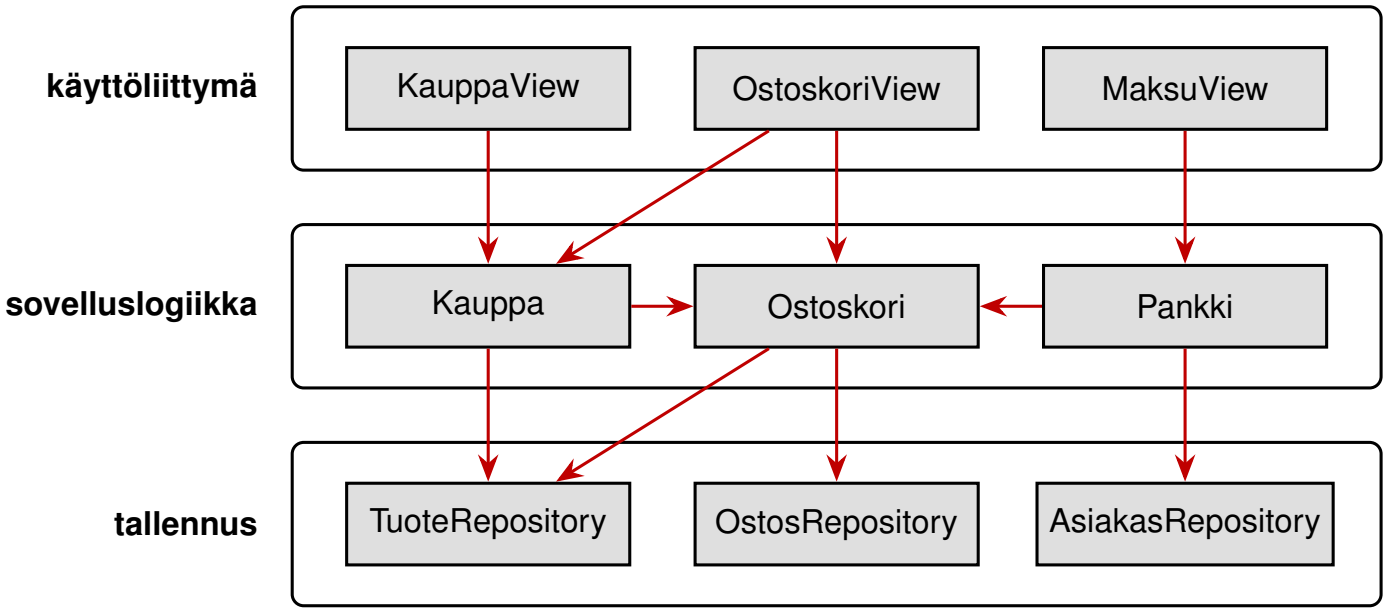
Ominaisuuspohjainen testaus (Property-based testing)

- Syötteitä ei valita käsin, vaan kuvataan ominaisuus, jonka pitää päteä kaikilla syötteillä

```
@given(st.integers(min_value=0, max_value=100),
       st.integers(min_value=-10, max_value=200))
def test_ottaminen_ei_hukkaa_tavaraa(alku_saldo, maara):
    varasto = Varasto(100, alku_saldo)
    otettu = varasto.ota_varastosta(maara)

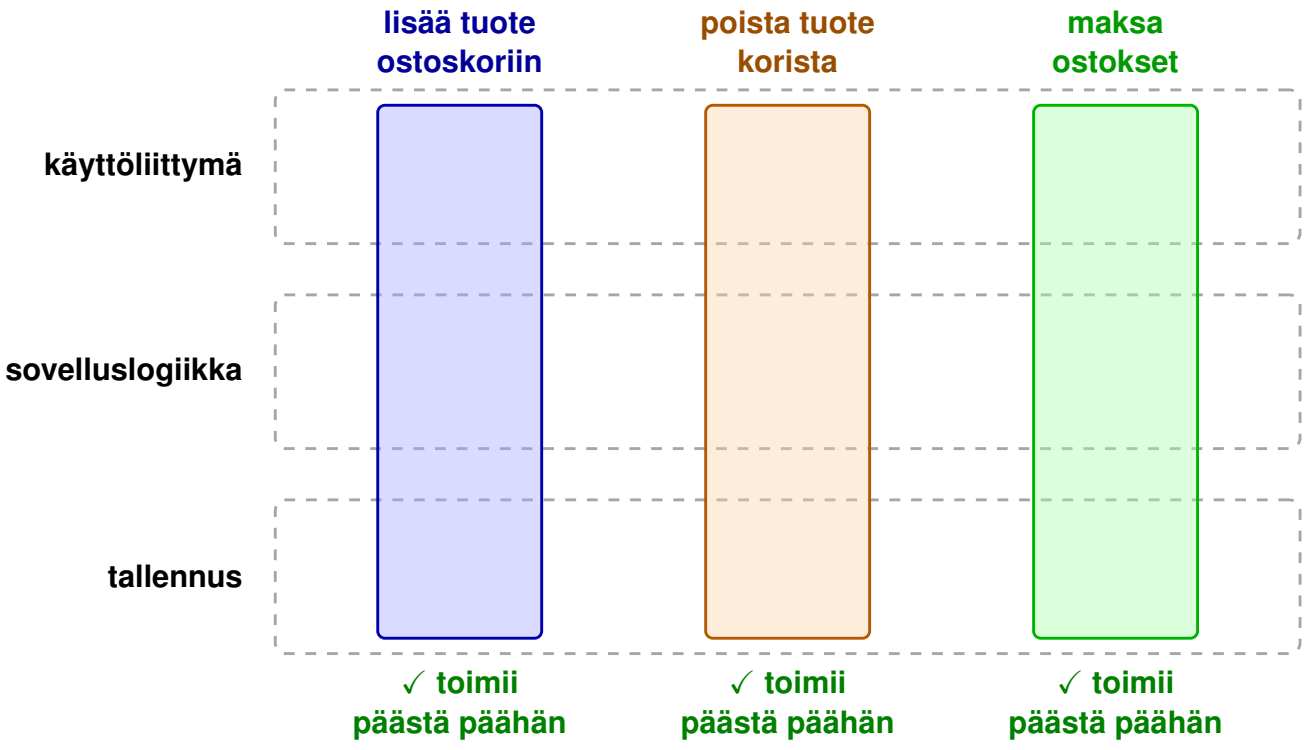
    assert varasto.saldo >= 0
    assert otettu + varasto.saldo == alku_saldo
```

Rakenteeseen perustuva integraatio

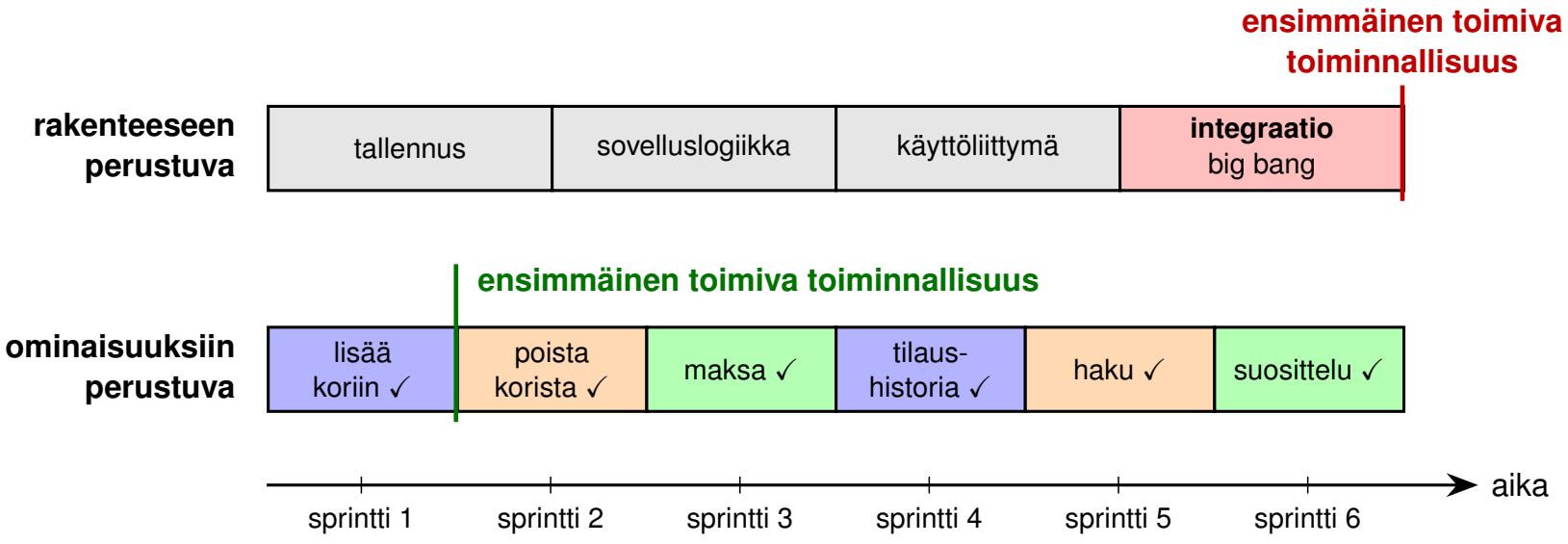


kaikki integroidaan kerralla vasta lopussa: toimiiko?

Ominaisuuksiin perustuva integraatio



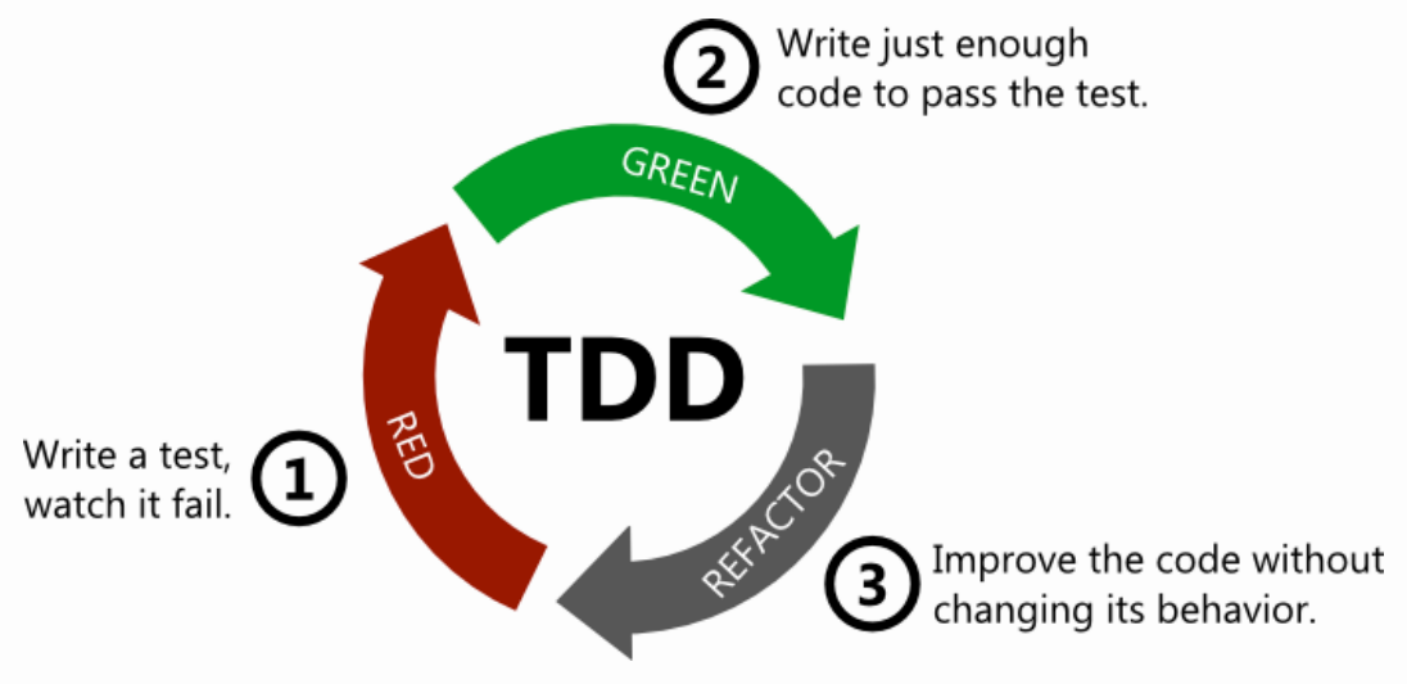
Milloin ensimmäinen toimiva toiminnallisuus?



Luento 6

10.11.2026

Test driven development (TDD)



1. Kirjoitetaan sen verran testiä että testi ei mene läpi
2. Kirjoitetaan koodia sen verran, että testi menee läpi
3. Jos huomataan koodin rakenteen menneen huonoksi refaktoroidaan koodin rakenne paremmaksi
4. Jatketaan askeleesta 1

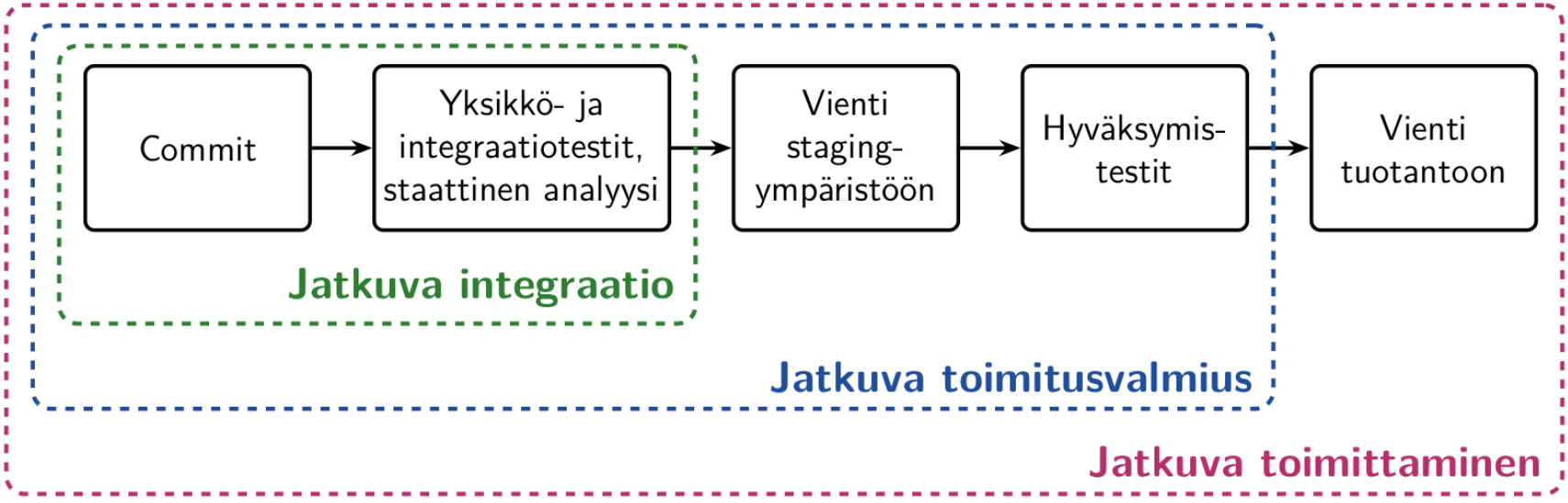
Testit asiakkan kielellä

```
*** Test Cases ***
Run | Debug | Run in Interactive Console
Login With Correct Credentials
    Set Username  kalle
    Set Password  kalle123
    Submit Credentials
    Login Should Succeed

Run | Debug | Run in Interactive Console
Login With Incorrect Password
    Set Username  kalle
    Set Password  wrong
    Submit Credentials
    Login Should Fail With Message  Invalid username or password
```

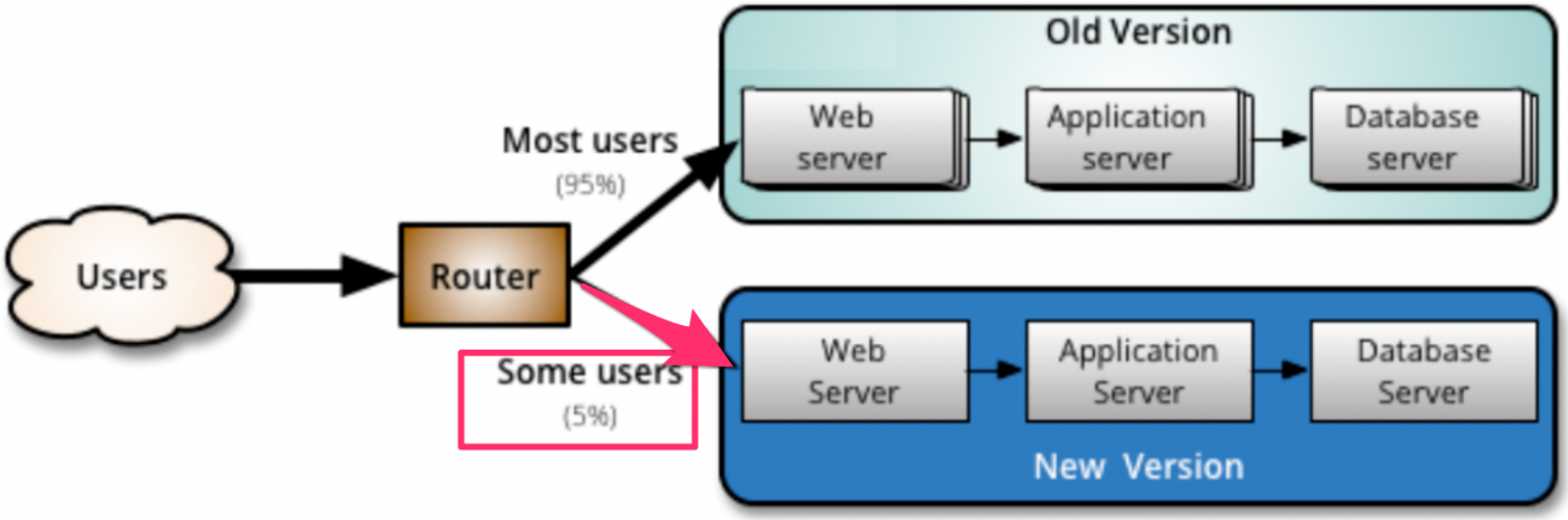
Deployment pipeline

- Vaiheet, joiden suorittaminen edellytetään, että commitattu koodi saadaan siirrettyä staging/tuotantoympäristöön



Canary release

- Kaksi rinnakkaista tuotantoympäristöä, joista uudet ominaisuudet viedään toiseen

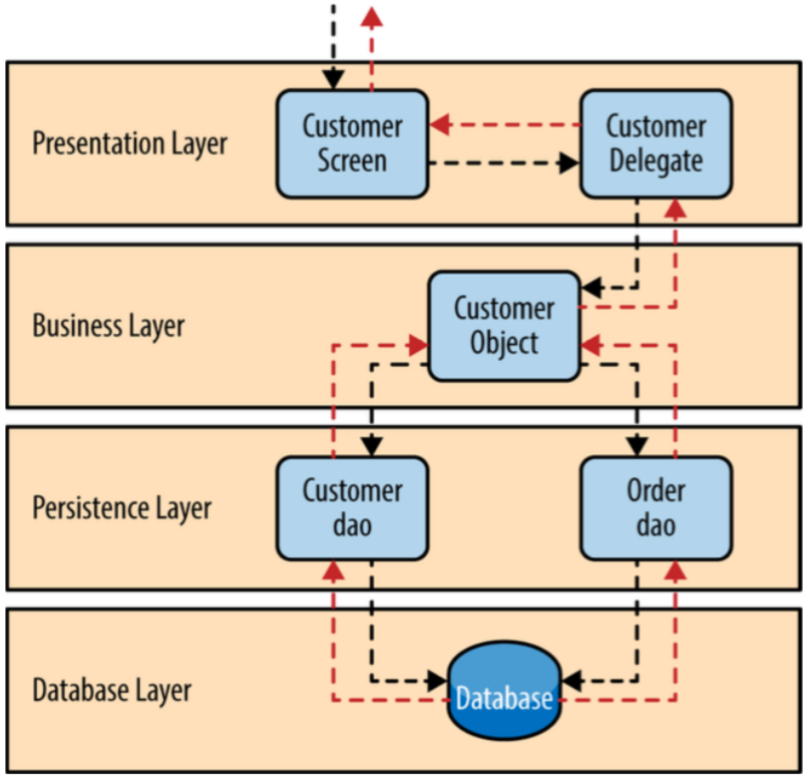


Luento 7

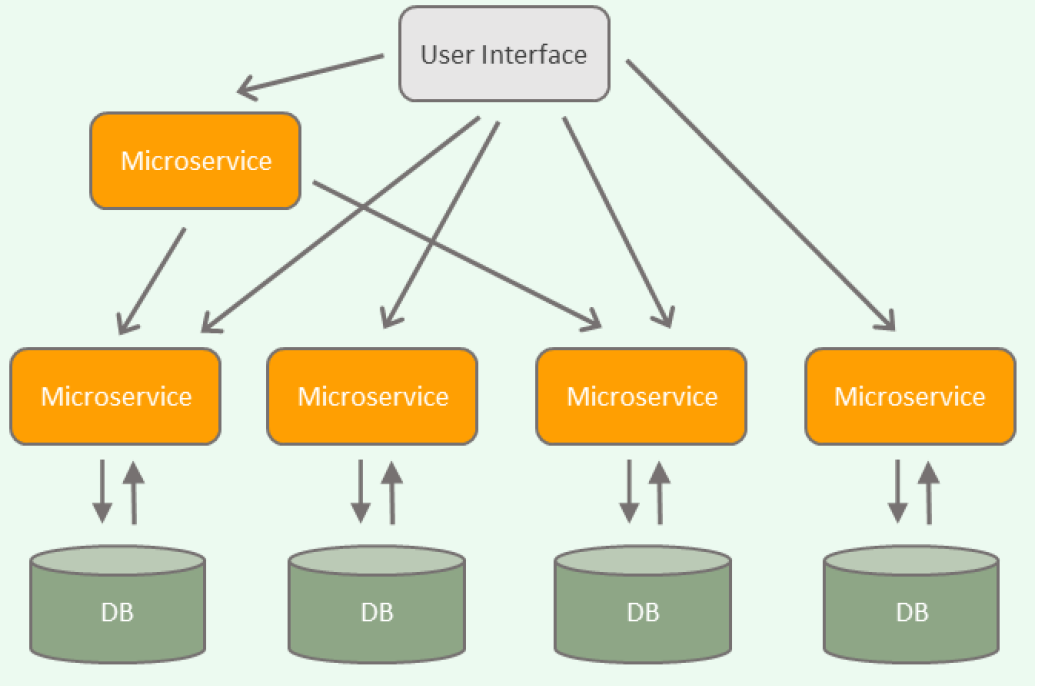
16.11.2026

Kerrosarkkitehtuuri

- *Kerros* on kokoelma toisiinsa liittyviä olioita, jotka muodostavat toiminnallisuuden suhteen loogisen kokonaisuuden



Mikropalveluarkkitehtuuri



- sovellus koostataan useista (jopa sadoista) pienistä verkossa toimivista autonomisista palveluista

Luento 8

17.11.2026

Laadukkaan koodin tuntomerkkejä

- Laadukkaalla koodilla joukko yhteneviä ominaisuuksia, tai *laatuattribuutteja*, esim. seuraavat:
  - kapselointi
  - korkea koheesion aste
  - riippuvuuksien vähäisyys
  - toisteettomuus
  - testattavuus
  - selkeys

Koheesio Flask-sovelluksessa

```python
@app.route("/create_todo", methods=["POST"])
def todo_creation():
    content = request.form.get("content")

    try:
        validate_todo(content)
        create_todo(content)
        return redirect("/")
    except Exception as error:
        flash(str(error))
        return  redirect("/new_todo")

# util.py

def validate_todo(content):
    if len(content) < 5:
        raise UserInputError("Todo content length must be greater than 4")

    if len(content) > 100:
        raise UserInputError("Todo content length must be smaller than 100")

# todo_repository.py

def create_todo(content):
    sql = text("INSERT INTO todos (content) VALUES (:content)")
    db.session.execute(sql, { "content": content })
    db.session.commit()
```

Laadukkaan koodin tuntomerkkejä

- Laadukkaalla koodilla joukko yhteneviä ominaisuuksia, tai *laatuattribuutteja*, esim. seuraavat:
  - kapselointi
  - korkea koheesion aste
  - riippuvuuksien vähäisyys
  - toisteettomuus
  - testattavuus
  - selkeys

Vibe-koodaus

Vibe-koodaus

- teknisen velan määrän kasvu

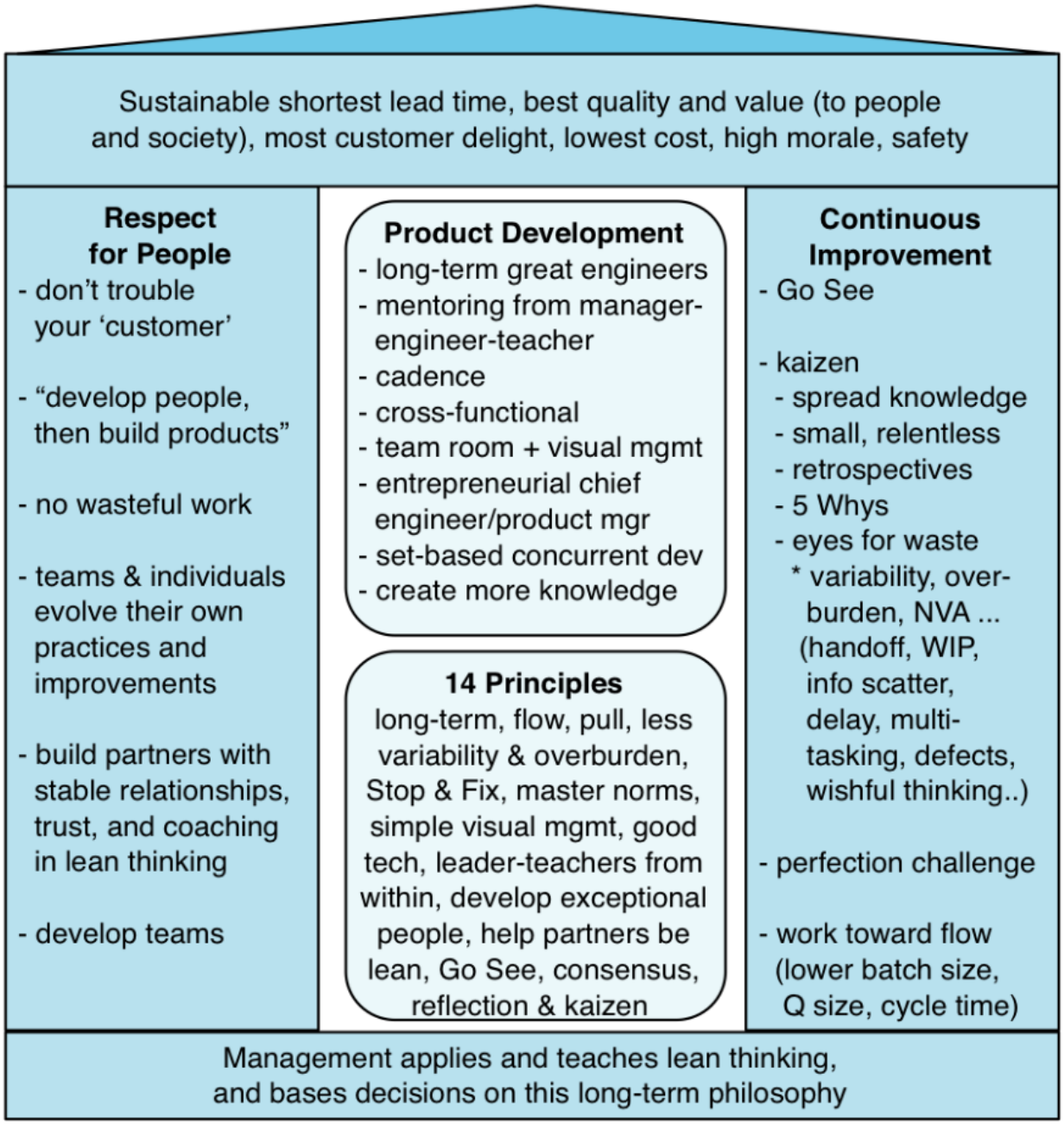
Vibe-koodaus

- teknisen velan määrän kasvu
- oppimisvelka

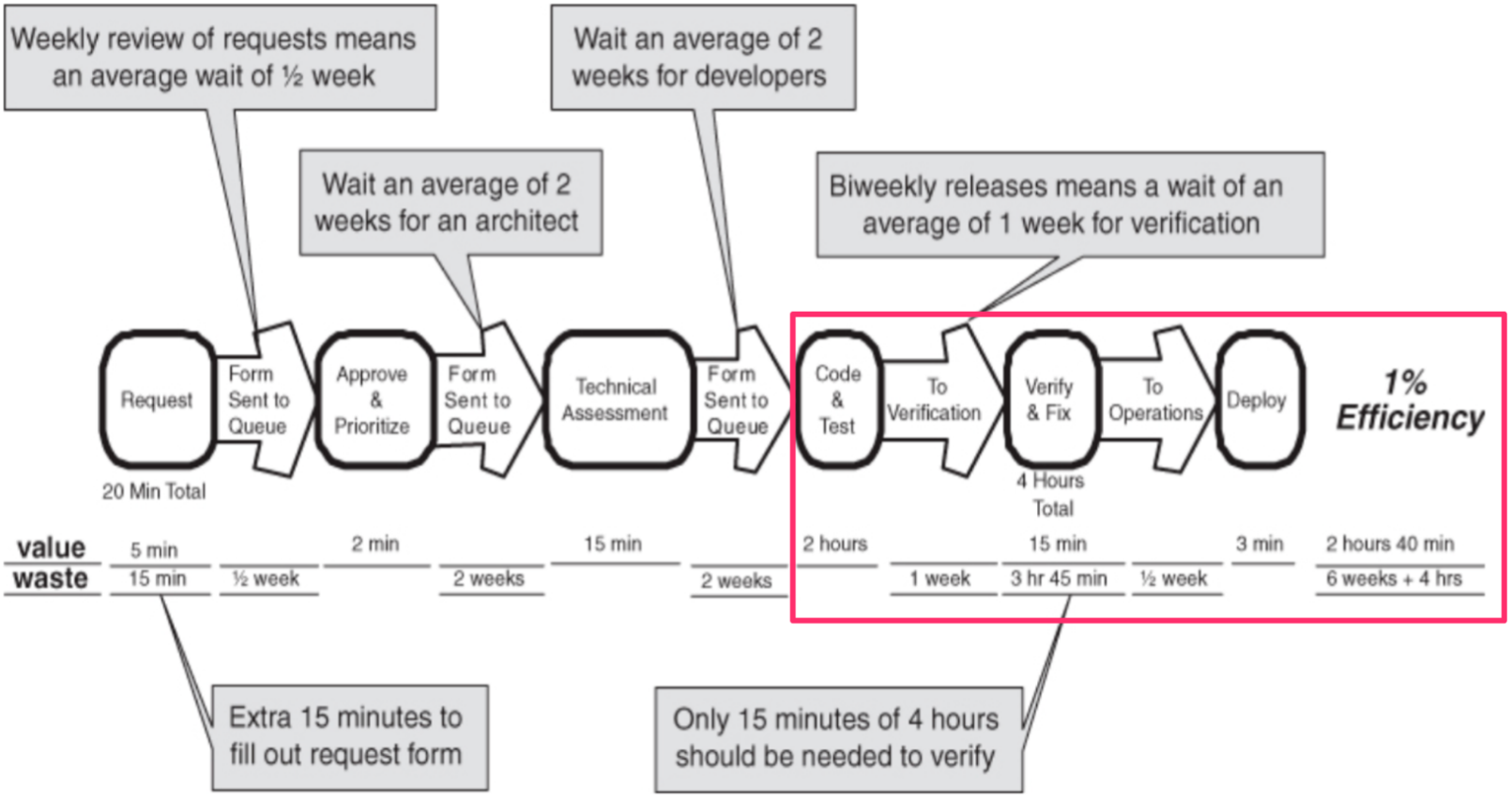
Luento 9

23.11.2026

Lean thinking house



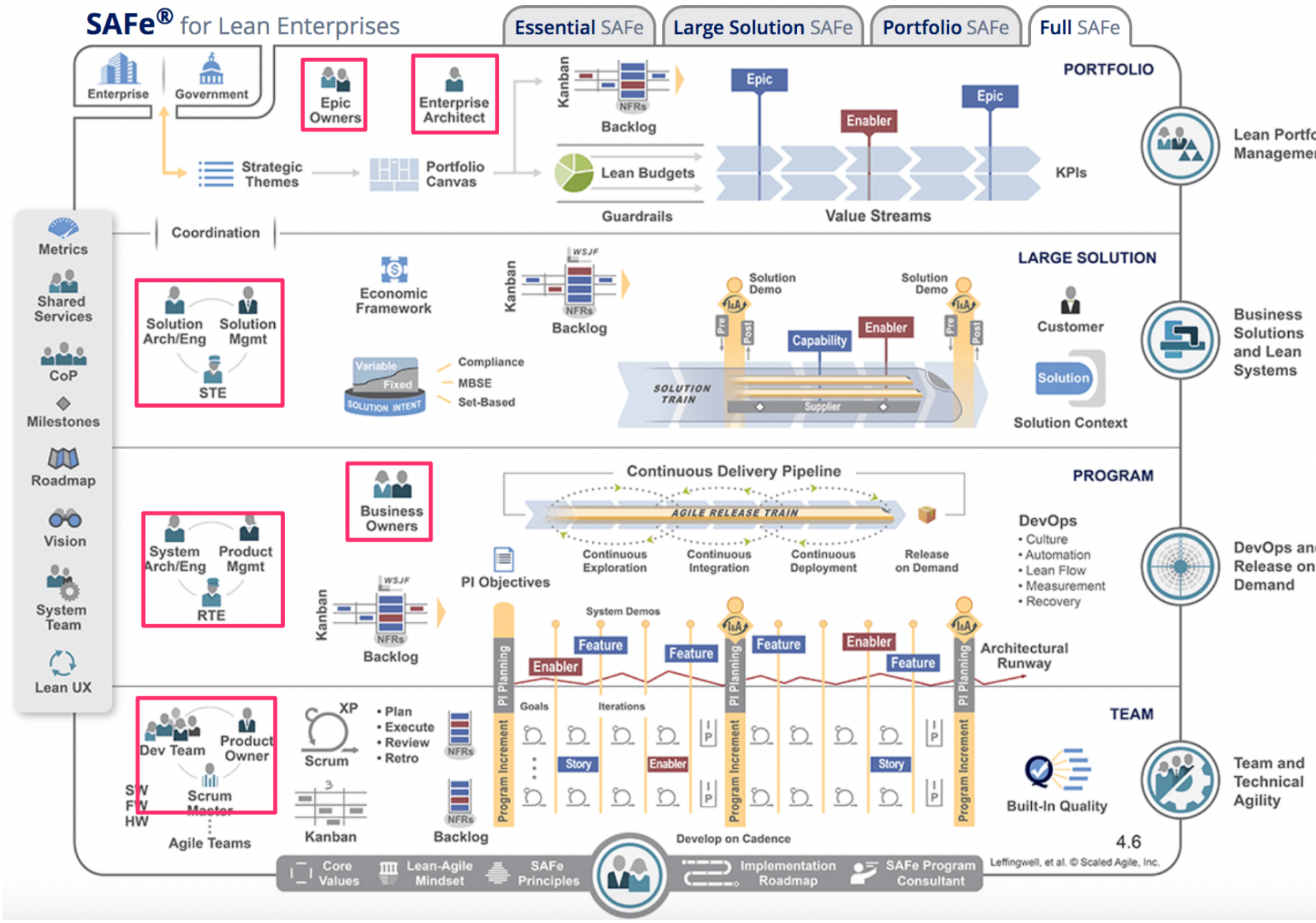
Perimmäisen syyn analyysi: five whys



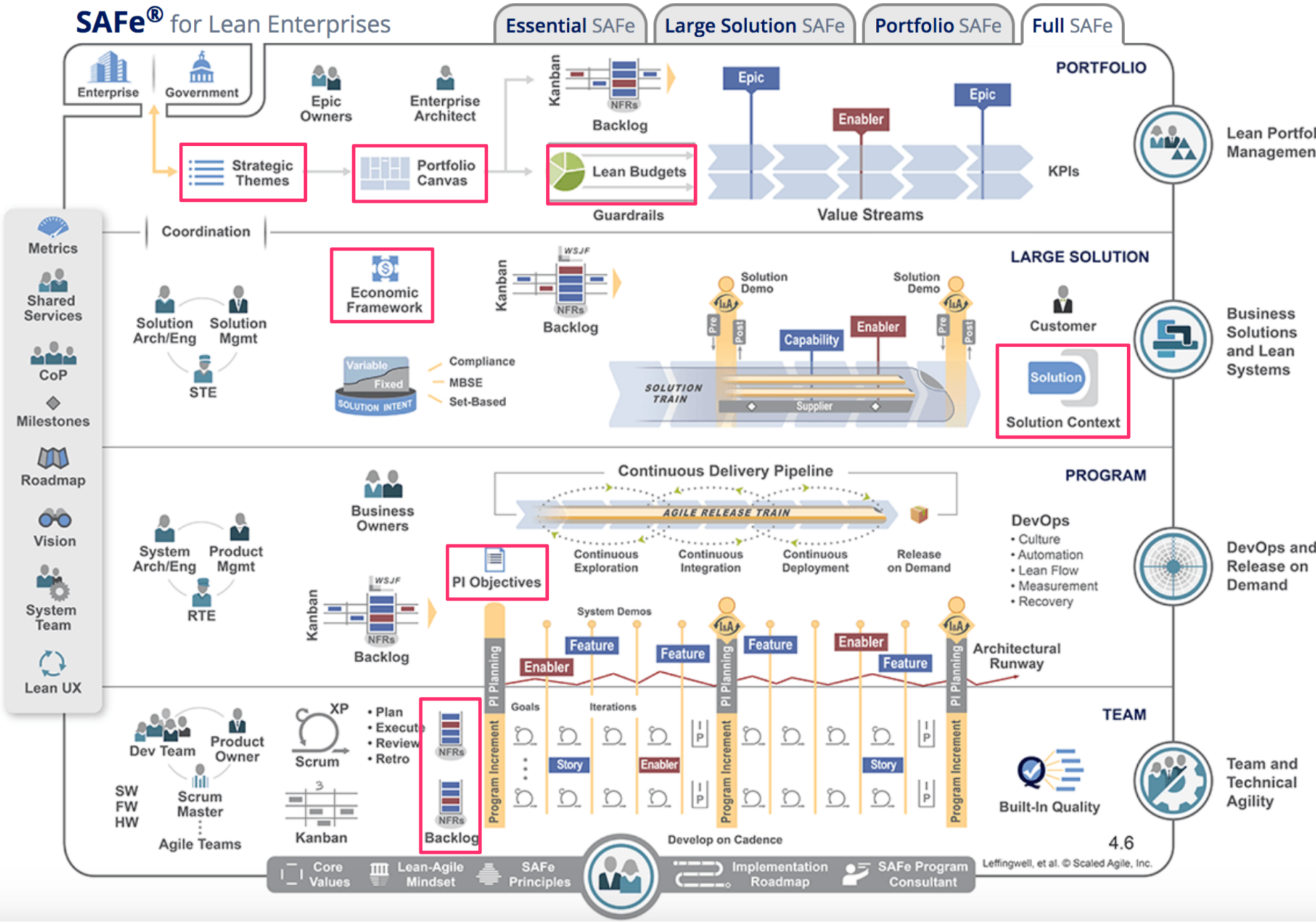
Luento 10

24.11.2026

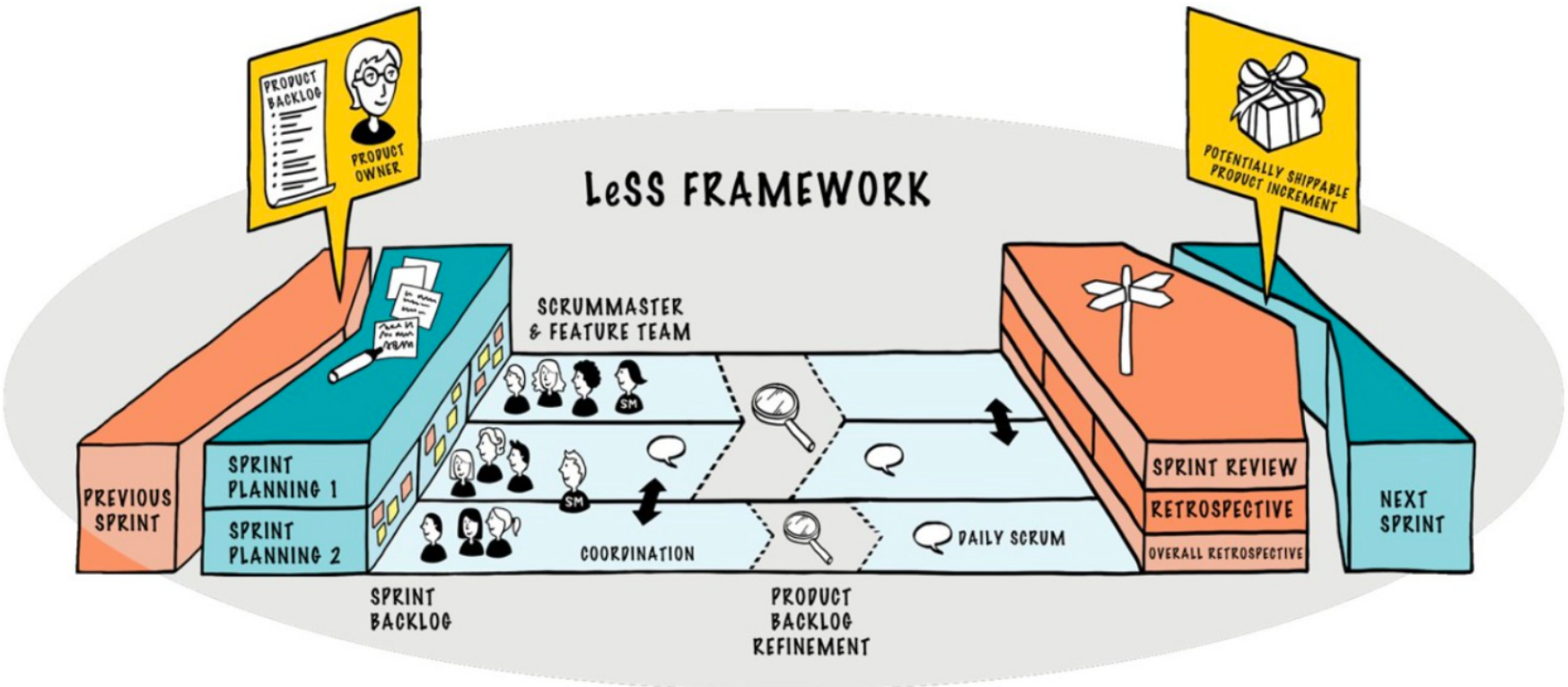
SAFe: paljon rooleja



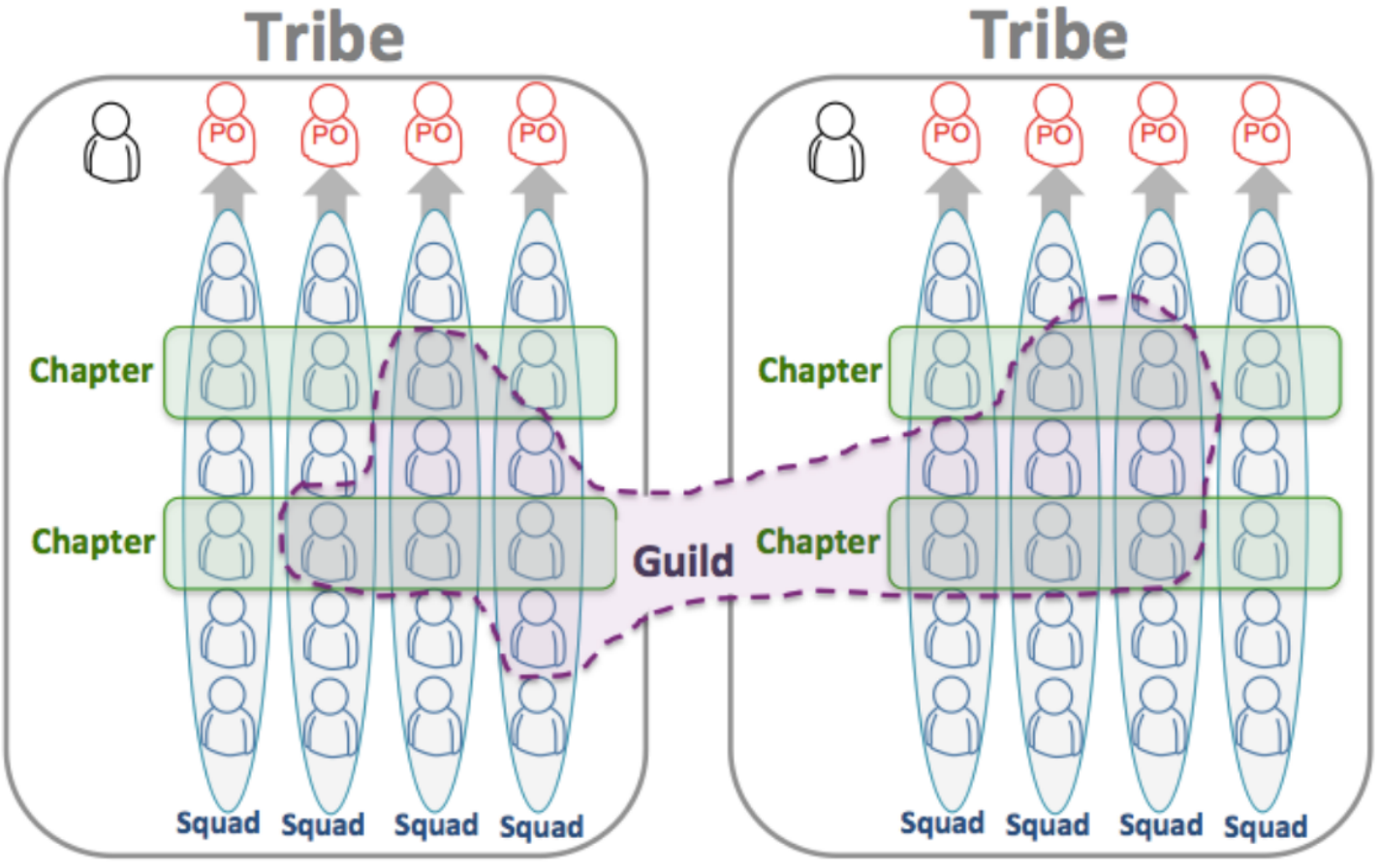
SAFe: paljon artefakteja



LeSS


PRODUCT BACKLOG
PRODUCT OWNER
LeSS FRAMEWORK
POTENTIALLY SHIPPABLE PRODUCT INCREMENT
SCRUMMASTER & FEATURE TEAM
SM
SM
PREVIOUS SPRINT
SPRINT PLANNING 1
SPRINT PLANNING 2
COORDINATION
DAILY SCRUM
SPRINT REVIEW
RETROSPECTIVE
OVERALL RETROSPECTIVE
NEXT SPRINT
SPRINT BACKLOG
PRODUCT BACKLOG REFINEMENT

Spotifyn malli



Tekniset käytänteet

